Streams with Python

nikki Streams · Python SDK guide

Skip to main content

Getting started

Setup Your Phone as an IoT Kit Streams with Python Streams with Node.js Streams with HTML + JavaScript Streams Library Overview Streams Library: Frequently Asked Questions (FAQ) Examples

Introduction — Streams: Your First IoT Kit Getting startedStreams with Python Streams with Python Use the official nikki-streams Python SDK to connect your Python program to the Streams app, receive phone sensor data, and control phone services.

The SDK is asynchronous and uses asyncio. It requires Python 3.10+ and a phone and computer on the same local network.

Official repository:

https://github.com/nikki-build/streams.python

Package:

https://pypi.org/project/nikki-streams/

1. Create Your Python Workspace

Create a small project:

streams-python/

```
├── main.py
└── README.md
```

Download the base template code Download the code (ZIP)

Create a virtual environment if you want to keep the project isolated:

python -m venv .venv

Activate it, then install the SDK.

2. Install the SDK

Install the official package:

pip install nikki-streams

Check for the latest version of the library. The SDK requires Python 3.10 or newer.

Check that it is installed:

pip show nikki-streams

3. Prepare the Streams App

Before running Python:

Create a Streams session. Join the session from your Android phone. Enable at least one sensor. Make sure the phone and computer are on the same local network. Find the WebSocket server address shown in the app. The app may show

something like:

ws://192.168.1.69:3000

For the Python SDK, pass the host and port without the ws:// prefix:

192.168.1.69:3000

The SDK automatically creates the WebSocket URL and adds the session ID.

4. Initialize the SDK

Import SDK and initialize it with your Streams Session ID:

```
from nikki_streams import SDK

SDK.initialize("YOUR_SESSION_ID")
```

The SDK only needs to be initialized once.

For example:

```
from nikki_streams import SDK

SESSION_ID = "YOUR_SESSION_ID"

SDK.initialize(SESSION_ID)
```

Keep your real Session ID private.

5. Create the Device

Create a device using the phone's address and a callback:

device = SDK.create_device(

```
    "192.168.1.69:3000",
    callback
```

)

The two important arguments are:

```
Argument    Purpose
Phone address   Tells the SDK where the Streams app is running
Callback    Function that receives events from the phone
```

The SDK automatically turns the address into a WebSocket URL containing your initialized session ID.

6. Add a Data Callback

The callback receives events from the phone.

A minimal callback is:

```
def callback(event):
    print(event)
```

Streams delivers connection status events as well as sensor data events.

A sensor event has this general structure:

```
CallbackPayload
├── type
└── data
```

For example, an incoming accelerometer event can look like:

```
CallbackPayload(
    type='data',
    data={
```

```
        'devID': '290b36b3-bbef-4478-88f7-32d7774bca99',
        'time': 1790078943647,
        'devName': 'my phone',
        'data':
        { 'Light': 388.6,
          'Accel': [0.02, -0.06, 9.81],
          'Gyro': [-0.0, -0.0, -0.0],
          'LinAccel': [0.0, 0.0, -0.0],
          'Mag': [90.88, 9.54, -99.62] }
        },
    error=None )
```

The type tells you what kind of event you received.

For sensor data:

```
event.type == "data"
```

Then the sensor information is available in:

```
event.data
```

7. Connect to the Phone

The SDK uses asyncio, so the connection is asynchronous.

```
await device.connect()
```

A complete minimal connection looks like this:

```
import asyncio

from nikki_streams import SDK

SESSION_ID = "YOUR_SESSION_ID"
PHONE_ADDRESS = "192.168.1.69:3000"

SDK.initialize(SESSION_ID)

def callback(event):
    print(event)

async def main():
    device = SDK.create_device(
        PHONE_ADDRESS,
        callback
    )

    await device.connect()

    await asyncio.sleep(10)

    await device.disconnect()
```

asyncio.run(main())

This follows the official SDK's basic example: initialize the SDK, create the device, provide a callback, connect, use the device, and disconnect.

8. Handle Sensor Data

The sensor readings are contained inside the nested data object.

'data': {

```
    'Light': 388.6,
    'Accel': [0.02, -0.06, 9.81],
    'Gyro': [-0.0, -0.0, -0.0],
    'LinAccel': [0.0, 0.0, -0.0],
    'Mag': [90.88, 9.54, -99.62]
```

}

The individual sensor values are represented as follows:

```
Sensor  Format  Example
Light   Single numeric value    388.6
Accel   [x, y, z]   [0.02, -0.06, 9.81]
Gyro    [x, y, z]   [-0.0, -0.0, -0.0]
LinAccel    [x, y, z]   [0.0, 0.0, -0.0]
Mag [x, y, z]   [90.88, 9.54, -99.62]
```

For example, the accelerometer values can be interpreted as:

```
accel = payload.data['data']['Accel']

x = accel[0]
y = accel[1]
z = accel[2]
```

which corresponds to:

```
x = 0.02
y = -0.06
z = 9.81
```

9. Understand Connection Events

The callback also receives connection status events.

Connected:

```
type = "statusChange"
data = "connected"
```

Disconnected:

```
type = "statusChange"
data = "disconnected"
```

You can handle them like this:

```
def callback(event):
    if event.type == "statusChange":
        print("Connection:", event.data)

    elif event.type == "data":
        print("Sensor:", event.data)
```

This gives your program one place to handle both connection state and incoming sensor data.

10. Control the Phone

The Python SDK can also send commands to supported phone control services.

The official SDK currently exposes:

Light control Volume control Haptic feedback Notifications [Source reference: turn1search1] Flashlight Turn the phone light on:

```
await device.set_light_status(True)
```

Turn it off:

```
await device.set_light_status(False)
```

The SDK sends the corresponding light command to the phone.

Volume Set media volume:

```
await device.set_volume(80)
```

The documented range is:

0 - 100

For example:

```
await device.set_volume(50)
```

Haptic Feedback Trigger vibration:

```
await device.set_haptic()
```

You can also provide a custom duration:

```
await device.set_haptic(500)
```

Notification Show a notification on the phone:

```
await device.set_notification(
    "Hello",
    "Welcome to Streams"

)
```

The first value is the notification title and the second is the notification text.

11. Complete Example

Here is a small example that connects to the phone, prints incoming events, controls the phone, waits, and disconnects.

```
import asyncio

from nikki_streams import SDK

SESSION_ID = "YOUR_SESSION_ID"
PHONE_ADDRESS = "192.168.1.69:3000"

SDK.initialize(SESSION_ID)

def callback(event):
    if event.type == "statusChange":
        print("Connection:", event.data)

    elif event.type == "data":
        print("Sensor data:", event.data)

async def main():
    device = SDK.create_device(
        PHONE_ADDRESS,
        callback
    )

    await device.connect()

    await device.set_light_status(True)
    await device.set_volume(60)
    await device.set_haptic()

    await device.set_notification(
        "Python SDK",
        "Connected successfully"
    )

    await asyncio.sleep(5)

    await device.set_light_status(False)

    await device.disconnect()
```

asyncio.run(main())

This combines the main pieces of the official Python SDK example.

12. The Python Workflow

Keep this mental model in mind:

Session ID

```
    ↓
SDK.initialize()
    ↓
SDK.create_device()
    ↓
```

Add callback

```
    ↓
device.connect()
    ↓
```

Receive events

```
    ↓
```

Process sensor data

```
    ↓
```

Send commands

```
    ↓
device.disconnect()
```

The important methods are:

```
Method  Purpose
SDK.initialize()    Set the Streams Session ID
SDK.create_device() Create a phone device connection
device.connect()    Connect to the phone
Callback    Receive events
device.set_light_status()   Control the flashlight
device.set_volume() Set media volume
device.set_haptic() Trigger vibration
device.set_notification()   Display a notification
device.disconnect() Close the connection
```

13. Your First Python IoT Experiment

Once the basic connection works, combine a sensor with a phone action.

For example:

Accelerometer

```
      ↓
```

Python callback

```
      ↓
```

Detect movement

```
      ↓
```

set_haptic()

```
      ↓
```

Phone vibrates

Or:

Ambient Light

```
      ↓
```

Python callback

```
      ↓
```

Light level is low

↓

set_light_status(True)

↓

Phone flashlight turns on

This is the basic IoT pattern:

Sense → Process → Act

Your phone provides the sensors and actions. The Python program provides the logic.

Previous Setup Your Phone as an IoT Kit Next Streams with Node.js

1. Create Your Python Workspace

Download the base template code

2. Install the SDK

3. Prepare the Streams App

Flashlight Volume Haptic Feedback Notification